DESENVOLVIMENTO MOBILE

ALEX ZACHARIAS

ALEX@EINSTEINLIMEIRA.COM.BR

CLASSES

- Classe é uma representação para um conjunto de objetos que compartilham atributos, métodos e possuem algum relacionamento ou características semelhantes;
- Uma classe especifica a estrutura do objeto, sem informar quais são seus valores
- Uma classe pode ser instanciada em vários objetos do mesmo tipo.
- Ex: a classe cachorro contem vários cachorros diferentes, onda cada um deles é uma instancia (objeto) da classe cachorro.

CLASSES

- Formato de uma classe:

```
class nome_da_classe( [parâmetros recebidos pela classe] ) {
        [atributos]
        [métodos]
}
```

- Os parâmetros são opcionais, uma classe pode ou não receber parâmetros.

ATRIBUTOS

- São características ou propriedades dos objetos
- São todos os dados que precisamos guardar sobre o objeto
- Atributos do objeto Marcos:
  - Nome: Marcos da Silva
  - Idade: 27
  - Sexo: masculino
  - Peso: 76
- Objetos devem incluir atributos que o especializem, quantos forem necessários.
  - Ex: RG, CPF, RA, etc.

ATRIBUTOS

- Os atributos são declarados fora dos métodos e são como variáveis globais podendo ser usadas em qualquer parte do programa;
- Exemplo:

```
class Produtos {
    var codigo = 0;
    var nome: String = "";
    var quantidade: Int = 0;
    var preco: Double = 0.0;

    //colocar aqui os metodos
}
```

- Os atributos precisam ser inicializados com um valor;
- O tipo do atributo é opcional como nas declarações de variáveis;

ATRIBUTOS PRIVADOS

- Um atributo pode ser privado (private), o que quer dizer que ele só pode ser acessado de dentro da própria classe.

```
class Produtos {
    var codigo = 0;
    var nome: String = "";
    var quantidade: Int = 0;
    private var preco: Double = 0.0;

}
```

- O atributo preço só pode ser acessado de dentro da classe, os outros atributos podem ser acessado de qualquer lugar do programa.

MÉTODOS

- São comportamentos que alteram o estado do objeto, retorna dados do objeto ou prestam serviços a outros objetos;
- Exemplo:
  - Reajustar_Salário
  - Obter_Salário_Liquido
  - Obter_Tempo_de_Trabalho

MÉTODOS

- Os métodos podem ter tipo de dado para retorno e receber parâmetros, mas ambos são opcionais:

```
class nome_da_classe
{
   [atributos – quando existir]
   fun nome_do_método( parâmetros ): tipo_de_retorno
}
```

MÉTODOS

```
class Produtos {
    var codigo = 0;
    var nome: String = "";
    var quantidade: Int = 0;
    private var preco: Double = 0.0;

    fun setaPreco( p: Double){
        preco = p;
    }
}
```

- O método setaPreco recebe um valor e coloca o mesmo no atributo preco que por ser private só pode ser acessado de dentro da classe.

MÉTODOS

- Se desejar acessar o valor de um atributo privado de fora da classe, é necessário criar um método para enviar esse valor:

```
class Produtos {
    var codigo = 0;
    var nome: String = "";
    var quantidade: Int = 0;
    private var preco: Double = 0.0;

    fun setaPreco( p: Double){
        preco = p;
    }
    fun retornaPreco(): Double{
        return preco
    }
}
```

OBJETOS

- É qualquer coisa existente no mundo real;
- Objetos são únicos, e possuem valores distintos. Um aluno por exemplo possui apenas um RA, e não existe outro aluno com o mesmo RA.
- A criação de um objeto a partir de uma classe é chamada de instanciação. É um processo semelhante a criação de uma variável, ao instanciar o objeto o Kotlin aloca a memoria que irá armazenar esse objeto.
- A classe representa um modelo abstrato que serve apenas para definir os métodos e atributos, os objetos são os modelos concretos baseados na classe.

OBJETOS - INSTANCIANDO

- Para instanciar um objeto em Kotlin, fazemos da seguinte forma:

  var nome_objeto: nome_classe = nome_classe(parâmetros)

- Os parâmetros são opcionais;

```
fun main() {

    var produto: Produtos = Produtos()

}
```

- Instanciamos um objeto produto da classe Produtos

OBJETOS

- Uma classe pode ser instanciada em quantos objetos forem necessários;
- Os objetos herdam os métodos e atributos da classe;
- Para acessarmos os métodos ou atributos de um objeto basta chamarmos os mesmos da seguinte forma:

```
nome_objeto.nome_atributo
nome_objeto.nome_metodo( parâmetros )
```

OBJETOS

```
fun main() {
    var arroz: Produtos = Produtos()
    var feijao: Produtos = Produtos()

    print("Digite o preço do arroz: ")
    var entrada: String = readln()
    arroz.setaPreco( entrada.toDouble())

    print("Digite o preço do feijão: ")
    entrada = readln()
    feijao.setaPreco( entrada.toDouble())

    println("O preço do arroz é: R$ ${String.format("%.2f", arroz.retornaPreco())}")
    println("O preço do feijão é: R$ ${String.format("%.2f", feijao.retornaPreco())}")
}
```

OBJETOS

```
String.format("%.2f", arroz.retornaPreco())
```

- A função String.format transforma um numero em uma string seguindo um determinado critério;
- O atributo "%.2f" esta definindo que o resultado deve ter duas casas decimais.

OBJETOS

- Note que os objetos arroz e feijão são entidades independentes, mesmo sendo instanciados da mesma classe (produtos);

- Podemos acessar os atributos da classe da mesma maneira que fazemos com os métodos, exceto quando definimos os mesmos como private, nesse caso só podem ser acessados de dentro da própria classe.

OBJETOS

- Para acessar atributos privados temos que criar métodos para alteração do valor e para visualizar seu valor;
- No nosso exemplo criamos os métodos setaPreco(double p) e retornaPreco() para atribuir ou ler um valor;
- Desta forma podemos proteger os atributos, podendo validar se o valor que esta sendo informado é um valor válido;
- Como exemplo vamos impedir que um valor zerado ou negativo seja gravado como sendo o valor do produto:

OBJETOS

```
class Produtos {
    var codigo = 0;
    var nome: String = "";
    var quantidade: Int = 0;
    private var preco: Double = 0.0;

    fun setaPreco( p: Double){
        if (p>0) {
            preco = p;
        }
    }
    fun retornaPreco(): Double{
        return preco
    }
}
```

EXERCÍCIOS

- Crie uma classe chamada Pessoas
- Cada pessoa tem nome, idade, peso e altura
- Essa classe ter métodos para as seguintes tarefas:
  - Mostrar nome
  - Mostrar idade
  - Mostrar peso
  - Mostrar altura
  - Calcular IMC (peso/(altura*altura))
- Faça uma função executável que:
  - Instancie dois objetos pessoas
  - Atribua a esses objetos nome, idade, peso e altura quaisquer
  - Mostre o nome, idade, peso, altura e o IMC calculado dessas pessoas
  - Baseando-se na tabela de IMC, escreva a situação da pessoa.

Cálculo IMC	Situação
Abaixo de 18,5	Abaixo do peso ideal
Entre 18,5 e 24,9	Peso normal
Entre 25,0 e 29,9	Acima do peso
Entre 30,0 e 34,9	Obesidade grau I
Entre 35,0 e 39,9	Obesidade grau II
40,0 e acima	Obesidade grau III